Constructor University Bremen

Natural Science Laboratory
Electronics

Fall Semester 2025

Lab Experiment 3 – Fourier Series and Fourier Transform

Author of the report Mr. Nikoloz Burduli

Experiment conducted by	: Mr. Nikoloz Burduli, Mr. Thushar Venkataramanaiah, Mr Jamal Salimov.
Place of execution	: Teaching Lab 53
Date of execution	: October 22, 2025

1. Introduction

The purpose of this experiment is to study different signals using their Fourier coefficients and to understand the concept of the Fourier transform. The experiment focuses on analyzing signals in both time and frequency domains, showing how any complex waveform can be expressed as a combination of simple sinusoidal components.

The practical part of the work involves simulating and implementing the Fast Fourier Transform (FFT) to observe the frequency content of signals efficiently. The obtained experimental and simulated results are compared and discussed. The experiment includes both continuous-time and discrete-time cases, providing a clear understanding of how signals can be represented and analyzed in the frequency domain.

1.1. Prelab Problem 1 – Decibels

1. Given $x(t) = 5\cos(2\pi 1000t)$,
   a. what is the signal amplitude and the V_{pp} voltage?

   The Signal Amplitude $V = 5\,V$ and the Peak to Peal Voltage $V_{pp} = 10\,V$

   b. what is the V_{rms} value of the provided signal?

$$V_{rms} = \frac{V}{\sqrt{2}} = \frac{5}{\sqrt{2}} = 3.54\,V$$

   c. what is the amplitude of the spectral peak in dBV_{rms}?

$$dBV_{rms} = 20\log_{10}(V_{rms}) = 20\log_{10}(3.54) = 10.97$$

2. For a square wave of 1 V_{pp} the voltage changes between -0.5 V and 0.5 V,
   a. what is the signal amplitude in V_{rms}?

$$V_{rms} = \sqrt{\frac{\int_0^T v(t)^2 dt}{t}} = \sqrt{\frac{0.5 \times (t)_0^T}{T}} = \sqrt{0.25} = 0.5\,V$$

   b. what is the amplitude in dBV_{rms}?

$$dBV_{rms} = 20\log_{10}(V_{rms}) = 20\log_{10}(0.5) = -6.02\,dB$$

1.2. Prelab Problem 2 – Determination of Fourier Series Coefficients

1. Determine the Fourier series coefficients up to the 5th harmonic of the function

$$f(t) = \frac{2}{T}t \qquad -\frac{T}{2} < t < \frac{T}{2}$$

To determine the Fourier series coefficients of the given function, its symmetry can be used to simplify the process. Since the function is odd, only sine terms will appear in its Fourier expansion, meaning that all cosine coefficients are zero. Therefore, $b_a = 0$. The total time period of the signal is $T = 1$, which gives the fundamental angular frequency as $\omega_0 = \frac{2\pi}{T} = 2\pi$.

$$b_n = \frac{2}{T}\int_{-\frac{T}{2}}^{\frac{T}{2}} f(t)\sin\left(\frac{2\pi nt}{T}\right)dt$$

$$b_n = \frac{2(-1)^{n+1}}{\pi n}$$

And the Fourier Series Coefficients up to the 5th harmonic:

$$b_1 = \frac{2}{\pi} \quad b_2 = -\frac{1}{\pi} \quad b_3 = \frac{2}{3\pi} \quad b_4 = -\frac{1}{2\pi} \quad a_5 = \frac{2}{5\pi}$$

2. Use MatLab to plot the original function and the inverse Fourier transform. Put both graphs into the same diagram.

For Part 2 of the experiment, a MATLAB script was used to generate both the original signal and its inverse Fourier transform (or Fourier series) representation. The resulting waveform obtained from the simulation is shown in the figure below.

```
t = -0.5:0.001:0.5;
w_0 = 2*pi;

F_t = 1/3;
for n = 1:5
    F_t = F_t + ((4*(-1)^n)/(n^2*pi^2))*cos(n*w_0*t); % fourier series representation
end

f_t = 4*t.^2; % original function

figure;
plot(t, f_t, 'r');
hold on;
plot(t, F_t, 'k--');
xlabel('Time (t)');
ylabel('f(t) and F(t)');
title('Original Function vs. Fourier Series Representation');
legend('f(t)', 'F(t)');
grid on;
```

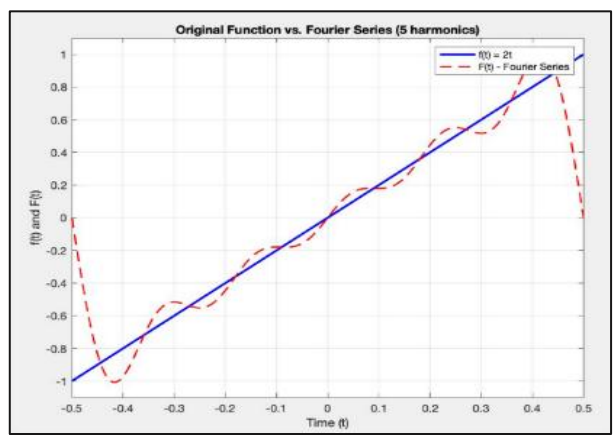


Figure 1. Plot of the original function and its Fourier Series

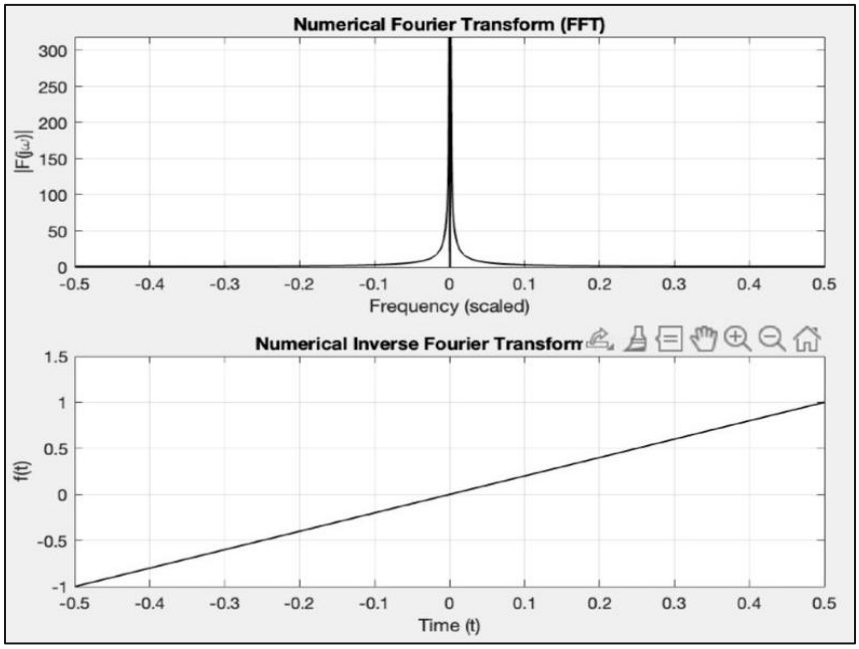


Figure 2. Plot of the Fourier transform and Inverse Fourier transform

1.3. Prelab Problem 3 – FFT of a Square/Rectangular Wave

1. Generate a square wave of 1ms period, 2Vpp amplitude, no offset, and duty cy cle 50% (hint : use 'square' function). Use 200kHz as the sampling frequency for the problem.

2. Plot the square wave in time domain.

3. Obtain the FFT spectrum using Matlab FFT function. Make the FFT length to be the length of the square wave data vector.

4. Plot the single-sided amplitude spectrum in dBVrms.

5. Plot the spectrum including only the first four harmonics in dBVrms. Hint: Use the Matlab command 'xlim'.

6. Repeat the previous steps using 20% and 33% duty cycles, respectively. Keep period and amplitude constant.

7. Discuss the changes for smaller pulse width. Use Eq. (7.11) to prove your statement.

Hint: Use the subplot command to plot the spectrum magnitude for the three cases 50%, 33% and 20% duty cycle to ease the comparison.

The MATLAB script shown below was written to perform the required calculations and generate the corresponding plots.

```
clc; clear;
T = 1e-3;
f = 1/T;                % Frequency of the square wave
V = 1;                  % Amplitude (1V, since it's centered at 0, +1V to -1V and 2Vpp)
fs = 200e3;             % Sampling frequency 200 kHz
t = 0:1/fs:20e-3;       % Time vector for one period
duty_cycles = [50, 33, 20];

figure;

for i = 1:length(duty_cycles)
    square_wave = V*square(2*pi*f*t,duty_cycles(i)); % Generating Square wave

    subplot(3, 3, 3*i - 2);
    plot(t, square_wave,'LineWidth',1.5);
    xlabel('Time (s)');
    ylabel('Amplitude (V)');
    title(['Square Wave with Duty Cycle ', num2str(duty_cycles(i)), '%']);
    xlim([0,5e-3]);

    L = length(t);
    Y = fft(square_wave, L); % Making the length of the FFT to the length of the square wave data vector
    P2 = abs(Y/L);
    L_half = floor(L / 2);
    P1 = P2(1:L_half + 1);
    P1(2:end-1) = 2*P1(2:end-1);
    f_axis = fs* (0:(L/2))/L;

    P1_dB = 20*log10(P1/sqrt(2));

    subplot(3, 3, 3*i - 1);
    plot(f_axis, P1_dB,'LineWidth',1.5); % Plotting single-sided amplitude spectrum in dB V_rms
    title(['Amplitude Spectrum in dBVrms with Duty Cycle ',
    xlabel('Frequency (Hz)');
    ylabel('Magnitude (dBVrms)');
    xlim([0 10*f]);

    subplot(3,3,3*i);
    plot(f_axis, P1_dB,'LineWidth',1.5);
    xlim([0 5*f]);  % Display only up to the fourth harmonic
    xlabel('Frequency (Hz)');
    ylabel('Amplitude (dBV_{rms})');
    title(['First Four Harmonics with (',num2str(duty_cycles(i)),'%) Duty Cycle)']);
end
```

Furthermore, based on the general expression for the Fourier coefficients, the coefficient c_k is proportional to $\frac{1}{k\pi}\sin(k\omega_0 T_1)$. As the pulse width T_1 becomes smaller, the value of $\sin(k\omega_0 T_1)$ also decreases for larger k, causing the higher harmonics to decay more quickly. This means that

higher-order harmonics contribute less to the overall waveform. In other words, when T_1 is reduced, the amplitude of the higher-frequency components decreases noticeably. This behavior can also be confirmed by observing the generated plot from the MATLAB simulation.

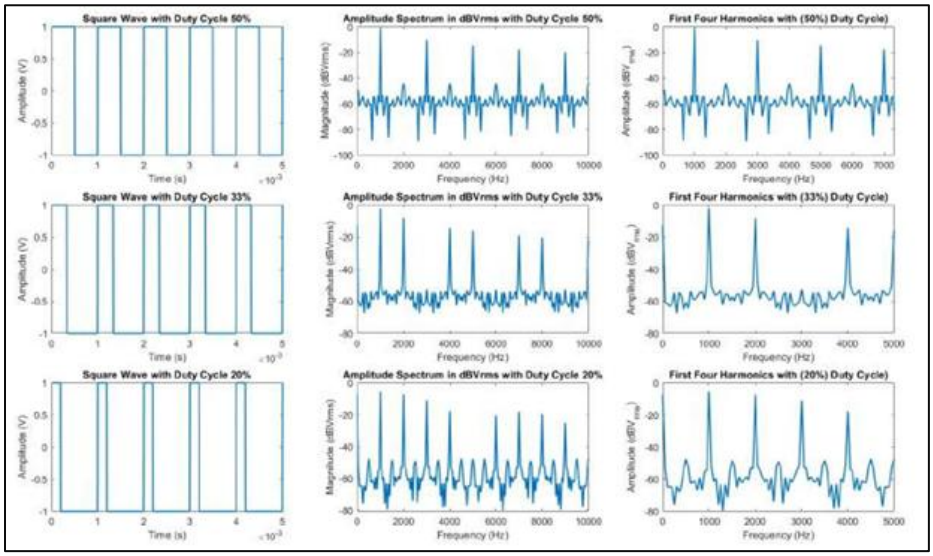


Figure 3. MATLAB Plots for 50%, 33% and 20% duty cycle and their FFT

1.4. Prelab Problem 4 – FFT of a Sound Sample

The Matlab command '[y,Fs] = audioread(filename)' reads a wave file specified by the string 'filename', returning the sampled data in 'y' and the sample rate in 'Fs' in Hertz.

1. Download the 'Soundfile for Prelab question 'FFT of a sound sample" from the course webpage.
2. Using Matlab, read the sound file and plot the first 10ms of the signal.
3. Use the Matlab FFT function to compute the spectrum and plot the single sided amplitude spectrum in dBVrms.
4. What are the tones forming this signal?

The MATLAB script below was created to read an audio signal and to plot both its waveform in the time domain and its frequency spectrum in the frequency domain.

```
%%
clc; clear;

filen = 's_samp.wav';

[y, Fs] = audioread(filen); % reading the sound signal
duration = 10e-3;
samples = round(5*duration*Fs);
signal_y = y(1:samples);

t = (0:samples-1)/Fs;

figure;
plot(t, signal_y); % Plotting the 10 ms of the signal
xlabel('Time (s)');
ylabel('Amplitude');
title('First 10 ms of the Signal');

L = length(t);
Y = fft(signal_y,L);
P2 = abs(Y/L);
L_half = floor(L/2);
P1 = P2(1:L_half+1);
P1(2:end-1) = 2*P1(2:end-1); % single sided amplitude
    spectrum
f_axis = Fs*(0:L_half)/L;

figure;
plot(f_axis, 20*log10(P1)); % plot of FFT
xlabel('Frequency (Hz)');
ylabel('Magnitude (dBVrms)');
title('Single-Sided Amplitude Spectrum');
```

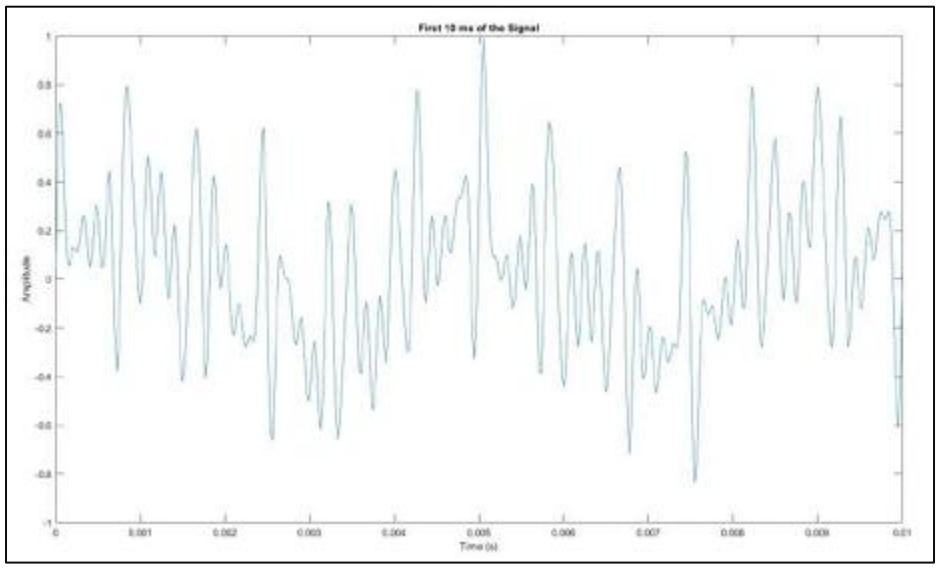


Figure 3. 10ms of the sound signal

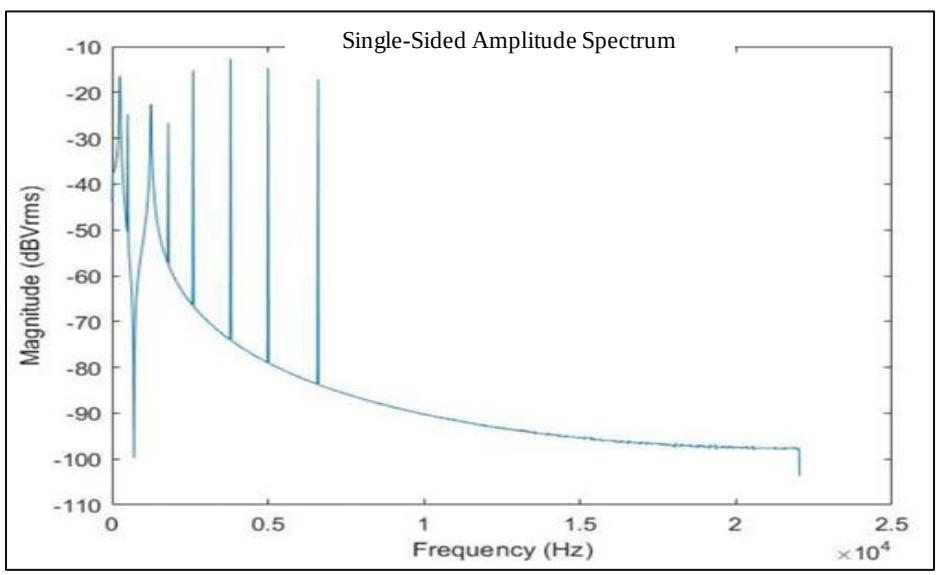


Figure 4. Single Sided FFT spectrum of the sound signal

The tones present in the signal were identified using MATLAB's pointer tool. The measured frequencies were approximately 260 Hz, 500 Hz, 700 Hz, 1240 Hz, 1800 Hz, 2600 Hz, 3800 Hz, 5000 Hz, and 6600 Hz. Among these, the tone observed to be pointing downward corresponded to the 700 Hz frequency.

2. Execution – Set-up and Results

Tools and Equipment

- 100µF and 2 x 1µF Capacitors
- 2 x 10 kΩ and 100 kΩ resistors
- Breadboard
- Oscilloscope for measurement
- Function Signal Generator
- 10 V DC Source

2.1. Execution Problem 1 – FFT of Single Tone Sinusoidal Wave

A sinusoidal signal with a frequency of 500 Hz, an amplitude of 2 Vpp, and no DC offset was generated using the function generator. The *Measure* function was then used to confirm these parameters, and the corresponding waveform hardcopy is provided below.

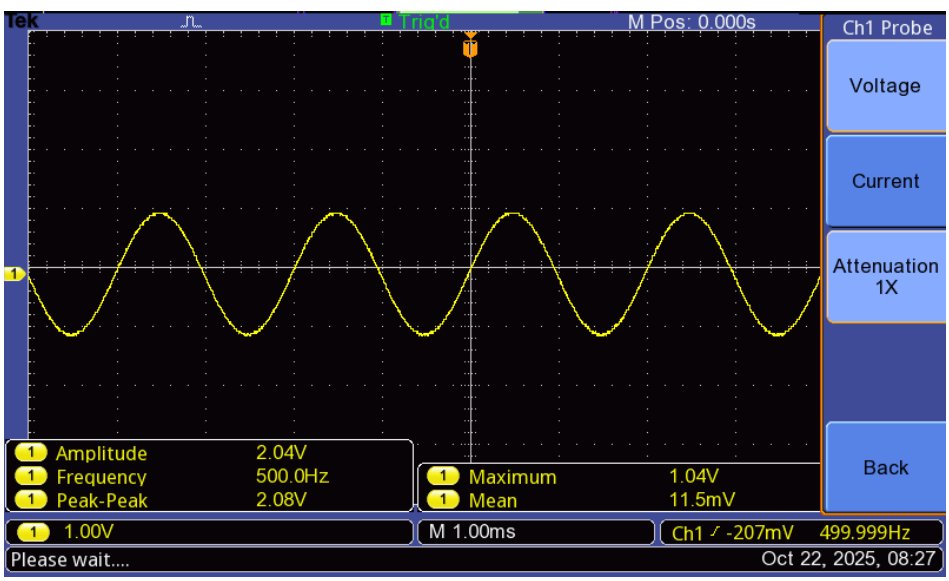


Figure 5. Sinusoidal Wave of frequency 500 Hz

The FFT spectrum of the signal was then obtained using the oscilloscope's FFT function. The key parameters of the spectrum were measured with the cursor tool on the oscilloscope, and hardcopies of both the full and zoomed-in spectra were captured for analysis.

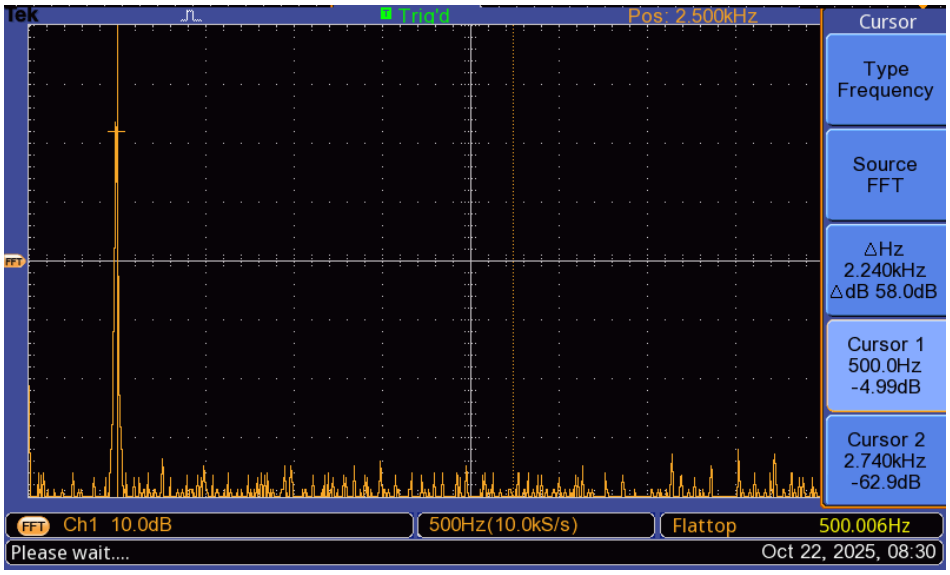


Figure 6. Overall FFT Spectra of the function

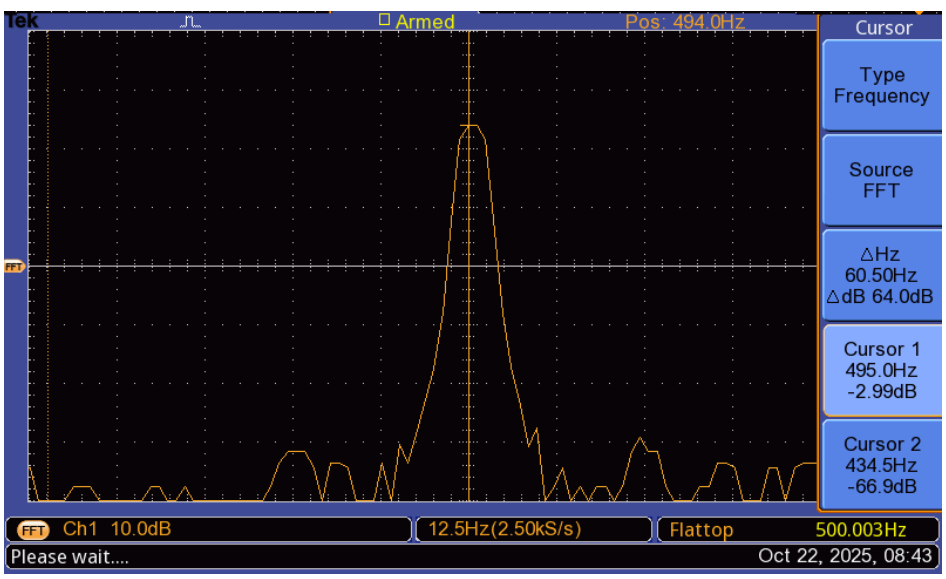


Figure 7. 10x zoomed spectra of the FFT

Similarly, a sinusoidal signal with a frequency of 2 kHz, a spectrum peak of 0 dB, and zero DC offset was generated. It is important to note that 0 dB corresponds to 1 V_{rms}. Using this value, the amplitude can be calculated as:

$$V_{rms} = \frac{V_{peak}}{\sqrt{2}} \quad \rightarrow \quad V_{peak} = 1 * \sqrt{2} = 1.4\ V$$

Since the peak-to-peak voltage is twice the amplitude,

$$V_{pp} = 2 * 1.4 = 2.8\ V$$

The resulting waveform corresponding to these parameters was then displayed on the oscilloscope.

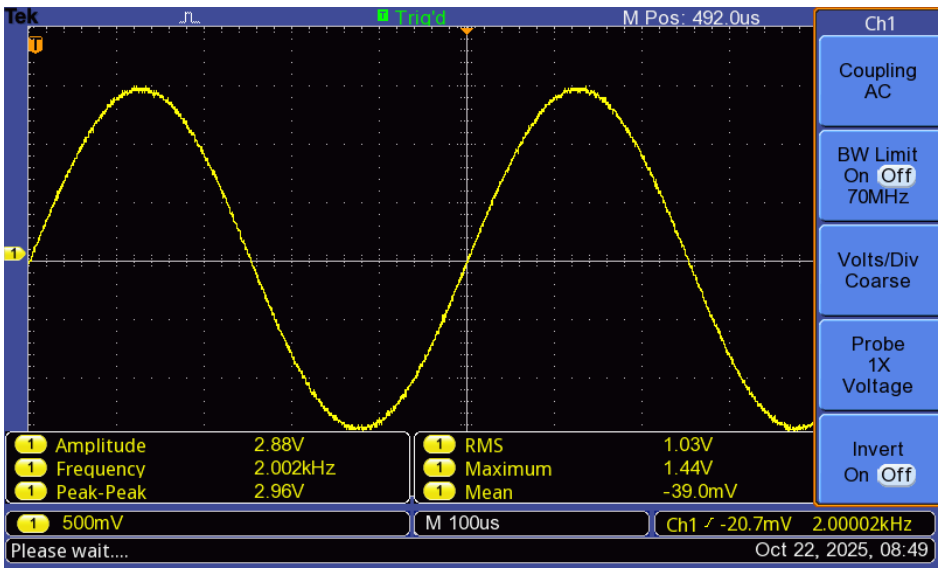


Figure 8. Sinusoidal Signal with 0dB peak, 2 kHz, 0V offset

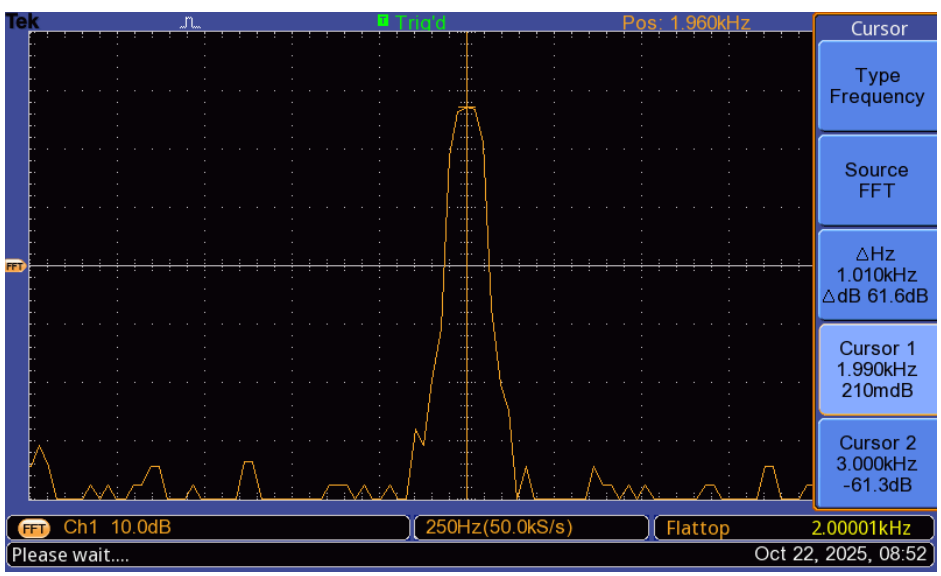


Figure 9. FFT of Sinusoidal Signal with 0dB peak, 2 kHz, 0V offset

The amplitude was measured with the cursor function and found to be approximately 210 mdB, which is very close to 0 dB, confirming the accuracy of the generated signal.

2.2. Execution Problem 2 – FFT of Square Wave

The function generator was then used to produce a square wave with a period of 1 ms (corresponding to a frequency of 1000 Hz), an amplitude of 2 Vpp, and no DC offset. The Measure function was utilized to verify the signal properties, and a hardcopy of the waveform was recorded for reference.

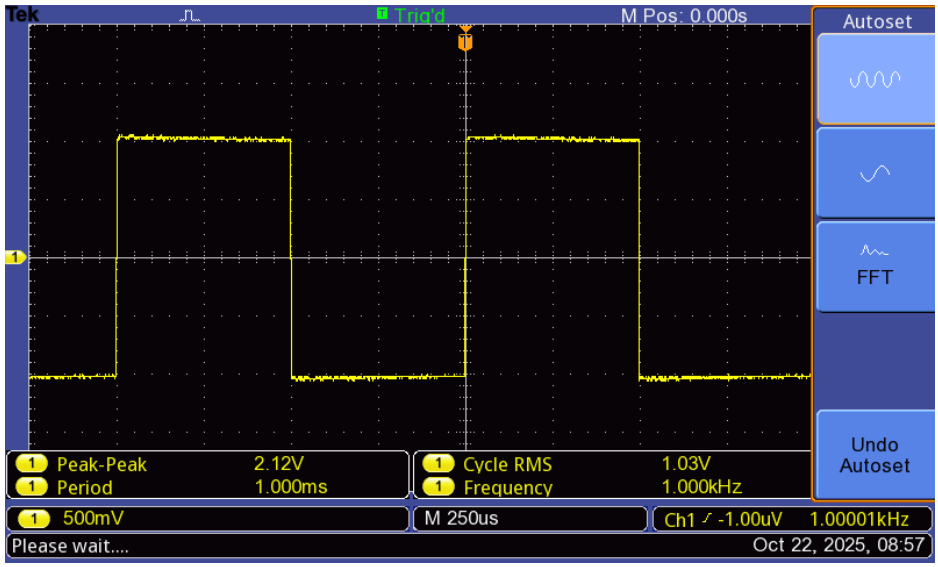
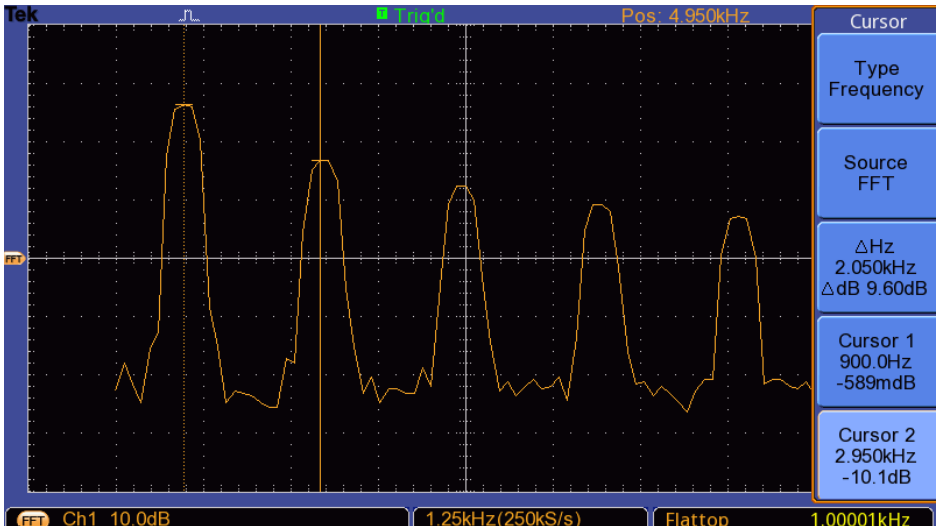


Figure 10. Square Wave of 1 kHz, 2 Vpp and no offset

The FFT spectrum of the square wave was then captured using the oscilloscope. The zoom control, set to a factor of 10, along with the cursor function, was used to measure the amplitude and frequency of the fundamental component and the first four harmonics. Hardcopies of each measurement were recorded for analysis.



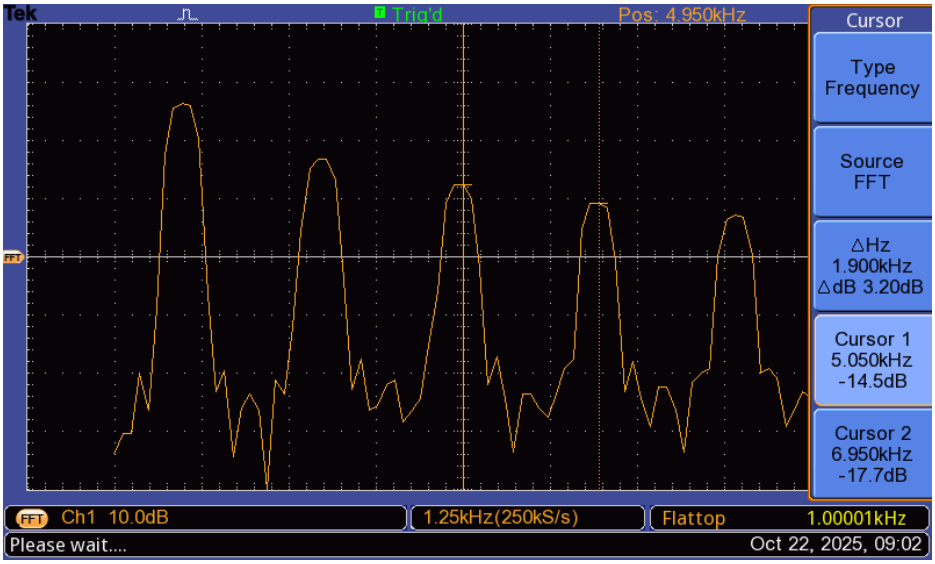


Figure 11. Second and third harmonic with respective amplitude and frequency

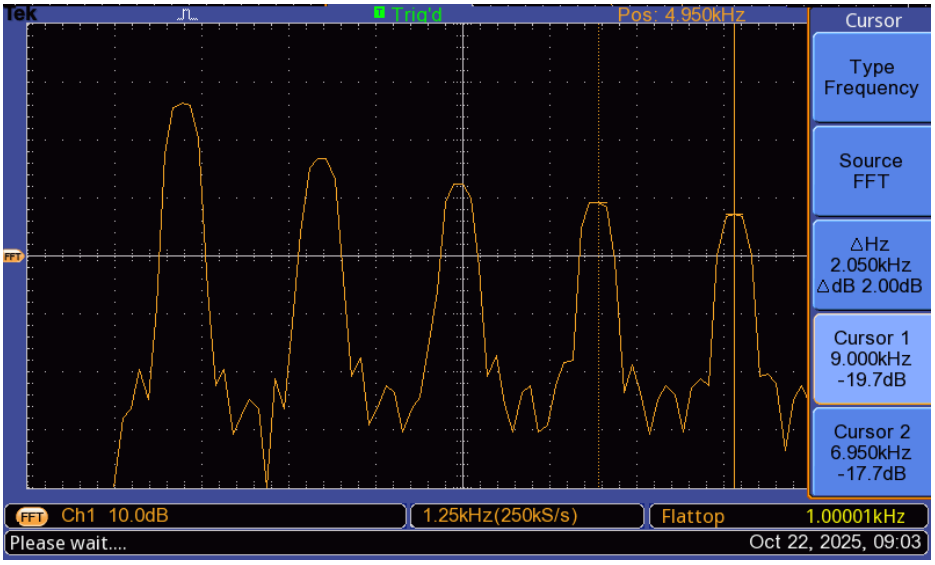


Figure 12. Fourth harmonic with respective amplitude and frequency

The measured amplitudes and frequencies of the fundamental and harmonic components are summarized in the table below.

Harmonic	Fundamental	First	Second	Third	Fourth
Frequency	900 Hz	2.950 kHz	5.050 kHz	6.950 kHz	9 kHz
Amplitude	-589 mdB	-10.1 dB	-14.5 dB	-17.7 dB	-17.7 dB

Table 1. Measured Frequency and Amplitude of fundamental and first four harmonic

The FFT spectrum for a 20% duty cycle square wave was then recorded, and the amplitudes and frequencies of the fundamental component and the first four harmonics were measured again. Hardcopies of these results were captured for reference.

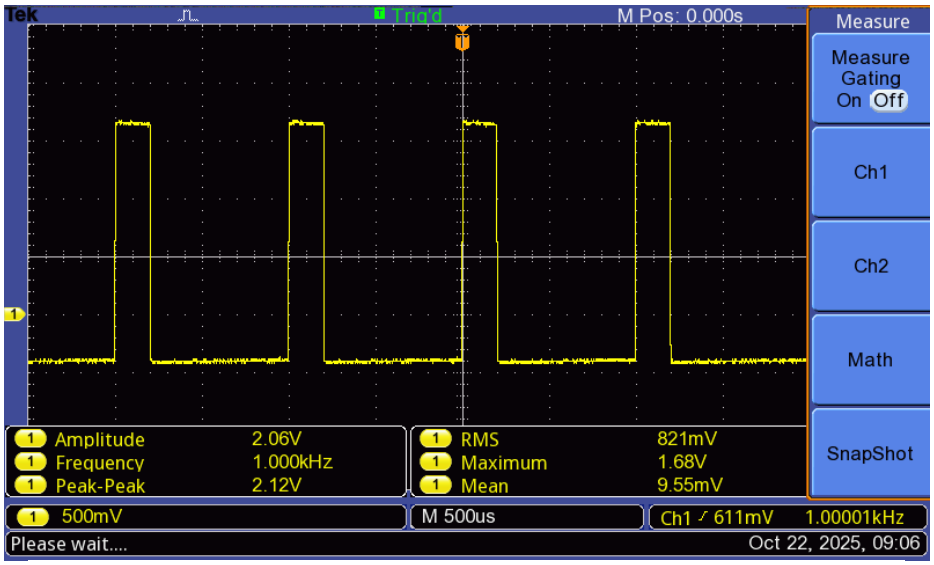


Figure 13. Time domain Plot of the square wave 20% duty cycle

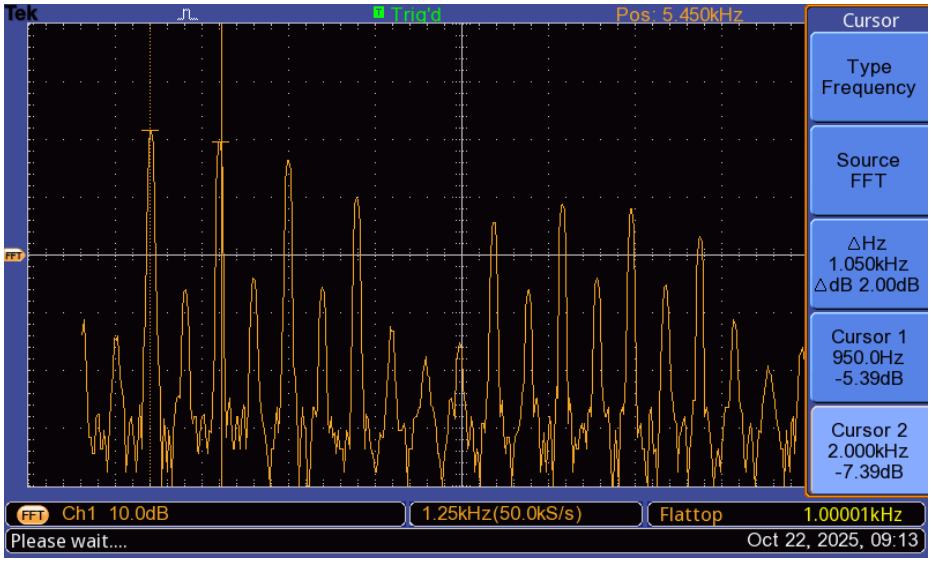


Figure 14. Fundamental and the first harmonic with their amplitude and frequency

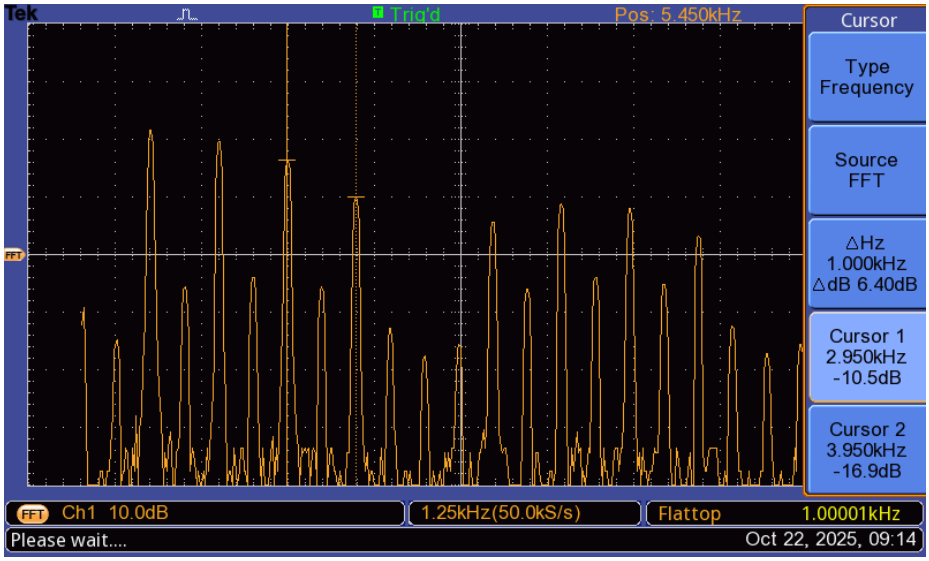


Figure 15. Second and third harmonic with their amplitude and frequency

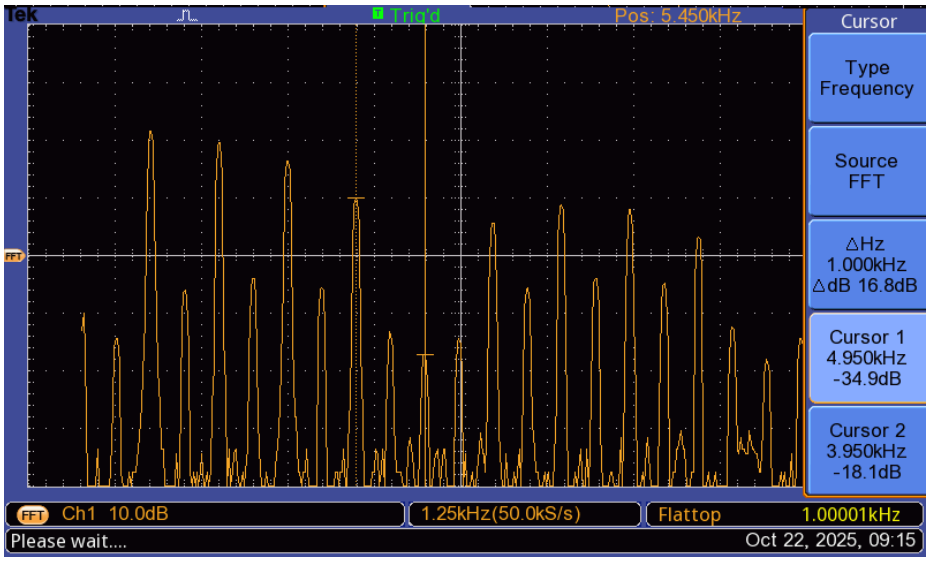


Figure 16. Fourth harmonic with its amplitude and frequency

The measured amplitudes and frequencies of the fundamental and harmonic components are summarized in the table below.

Harmonic	Fundamental	First	Second	Third	Fourth
Frequency	950 Hz	2 kHz	2.950 kHz	3.950 kHz	4.950 kHz
Amplitude	-5.39 mdB	-7.39 dB	-10.5 dB	-16.9 dB	-34.9 dB

Table 2. Measured Frequency and Amplitude of fundamental and first four harmonic

2.3. Execution Problem 3 – FFT of Multiple – Tone sinusoidal wave

For the final part of the experiment, the circuit was constructed using the sine output from the auxiliary signal generator together with a 2 Vpp, 10 kHz sinusoidal signal from the Agilent signal generator. The circuit was connected according to the diagram shown below.

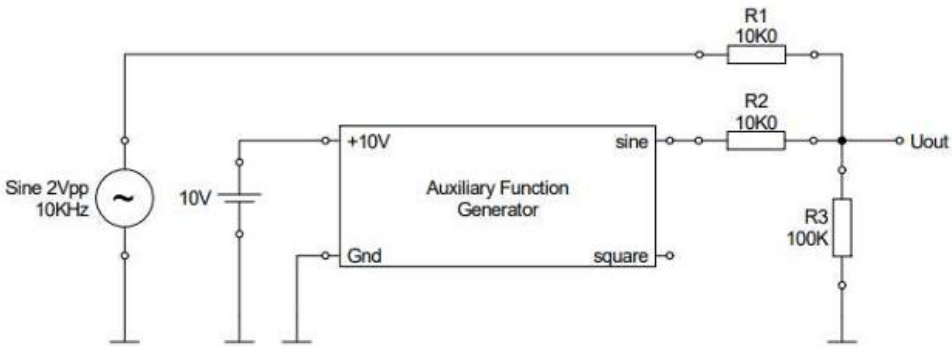


Figure 16. Circuit for the Problem 3

Hardcopies of the signal in the time domain and its corresponding FFT spectrum were captured for analysis.

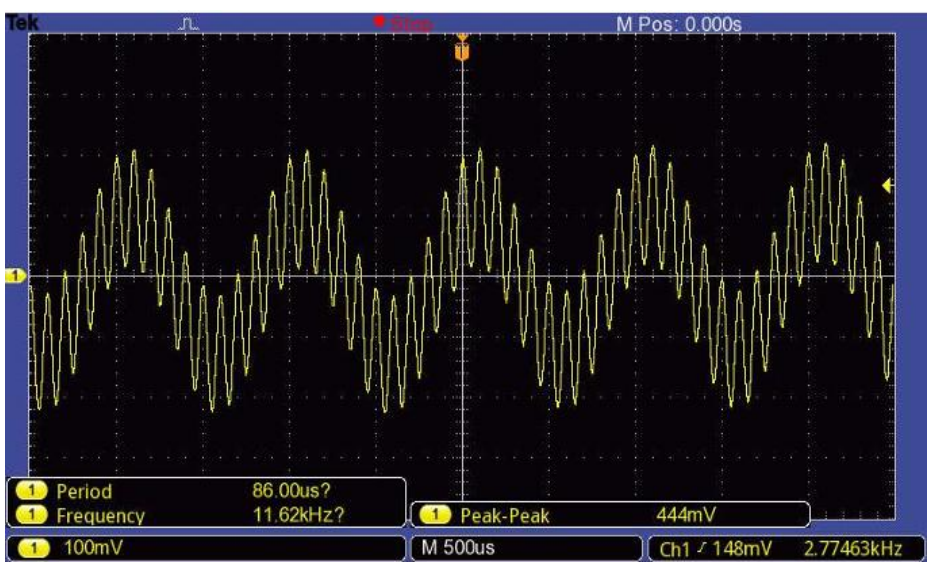


Figure 18. The signal in the time domain

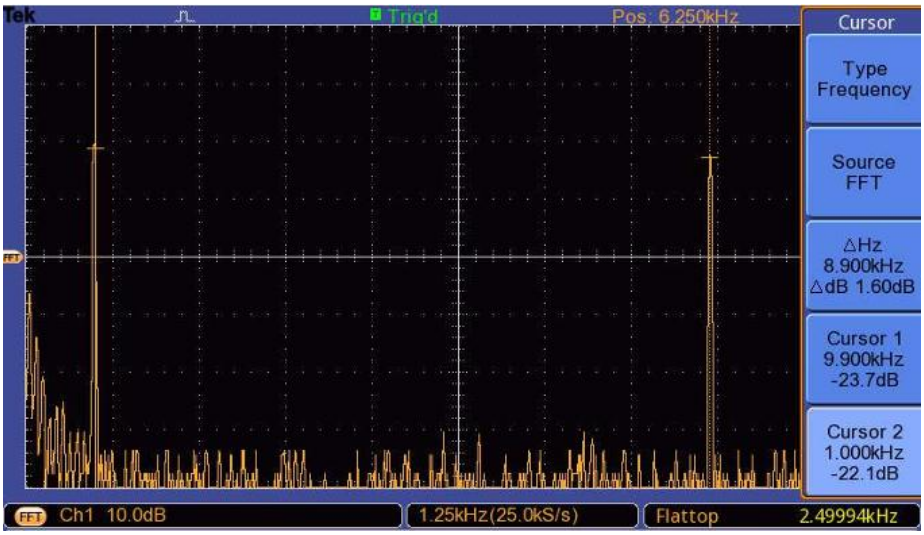


Figure 19. FFT spectrum of the signal and peaks measurement

3. Evaluation

3.1. Evaluation Problem 1 – FFT of Single – Tone sinusoidal wave

For the first part of the evaluation, as stated in the Execution Problem 1 section, the oscilloscope reference value of 0 dB corresponds to 1 V rms. Using the parameters provided in the manual, the FFT spectrum was generated in MATLAB, and the corresponding script is shown below.

```
clc; clear;
%%
fs = 5000;          % Sampling frequency
f = 500;            % Signal frequency
V = 1;              % Signal amplitude
L = 1500;           % Length of signal
t = (0:L-1)*(1/fs);    % Time vector

% Generate 500 Hz sinusoidal wave
sin_wave = V * sin(2 * pi * f * t);

% Compute FFT
Y = fft(sin_wave);     % FFT of the signal

% Two-sided spectrum (normalized)
P2 = abs(Y/L);              % Normalize FFT by length
L_half = floor(L/2);        % Half the length (for single-
   sided spectrum)
P1 = P2(1:L_half + 1);    % Single-sided spectrum
P1(2:end-1) = 2*P1(2:end-1);  % Compensate for single-sided
    FFT

% Frequency axis for plotting
f_axis = fs * (0:(L_half))/L;

% Convert to dB (20*log10)
P1_dB = 20 * log10(P1/sqrt(2));

% Plot FFT in dB
figure;
plot(f_axis, P1_dB);
title('Single-Sided Amplitude Spectrum (in dB)');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
grid on;
```

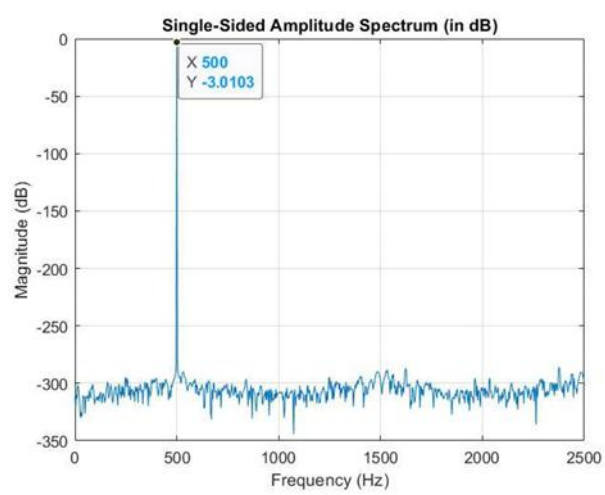


Figure 20. FFT spectrum of the signal with 500 Hz, 2 Vpp, no offset

The amplitude value at the peak was measured to be −3.0103 dB, and the corresponding frequency was 500 Hz. The measured amplitude showed some deviation from the calculated value. These results are summarized in the table below to allow comparison and to discuss potential inaccuracies and errors that may have occurred during the measurement process. Similarly, for the parameters provided in the manual, the FFT spectrum was obtained using MATLAB, and the corresponding script is shown below.

```
clc; clear;
%%
fs = 5000;          % Sampling frequency
f = 2000;           % Signal frequency
V = sqrt(2);        % Signal amplitude
L = 1500;          % Length of signal
t = (0:L-1)*(1/fs);    % Time vector

% Generate 2000 Hz sinusoidal wave
sin_wave = V * sin(2 * pi * f * t);

% Compute FFT
Y = fft(sin_wave, L);     % FFT of the signal

% Two-sided spectrum
P2 = abs(Y/L);
L_half = floor(L/2);      % Half the length (for single-sided spectrum)
P1 = P2(1:L_half + 1);    % Single-sided spectrum
P1(2:end-1) = 2*P1(2:end-1);  % Compensate for single-sided FFT

% Frequency axis for plotting
f_axis = fs * (0:(L_half))/L;

% Convert to dB (20*log10)
P1_dB = 20 * log10(P1/sqrt(2));

% Plot FFT in dB
plot(f_axis, P1_dB);
title('Single-Sided Amplitude Spectrum (in dB)');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
grid on;
```

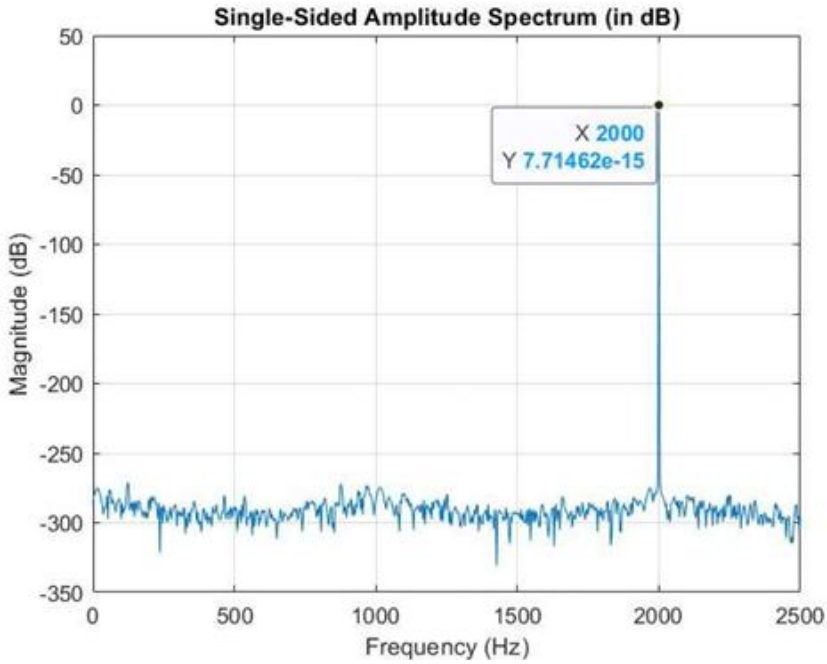


Figure 21. FFT spectrum of the signal with 0 dB spectrum peak, 2 kHz frequency and no offset

The amplitude value at the peak was measured to be extremely small, with a corresponding frequency of 2000 Hz. The measured amplitude was found to be consistent with the calculated value. The results are summarized in a table to compare the differences between the measured and calculated values for this and the previous case.

Signal	500 Hz		2000 Hz	
Quantity	Frequency	Amplitude	Frequency	Amplitude
Measured	500 Hz	-4.99 dB	1990 Hz	210 mdB
Calculated	500 Hz	-3.0103 dB	2000 Hz	7.71e-15 dB

Table 3. Measured and calculated values

For the 500 Hz frequency, the measured amplitude was −4.99 dB, whereas the calculated amplitude was −3.01 dB, showing a significant difference. This discrepancy is likely due to the measurement being taken from the overall spectrum rather than a zoomed-in view, resulting in lower resolution and reduced accuracy.

In contrast, the properties of the measured spectrum for the 2000 Hz signal were consistent with the calculated values. However, the measured amplitude was 210 m dB instead of the expected 0 dB. This slight offset can be attributed to the inherent noise of the oscilloscope. Even without an applied signal, internal electronic noise may produce a small non-zero reading, leading to a minor positive offset in the FFT amplitude and a small deviation from the theoretical 0 dB value.

3.2. Evaluation Problem 2 – FFT of Square Wave

To accurately measure the frequency components in the frequency domain, the frequency scale was expanded using the time base (sec/div) control. Expanding the frequency scale effectively decreases the bandwidth, which increases resolution and allows finer observation of the signal, resulting in more accurate measurements.

Changing the duty cycle affected both the time-domain and frequency-domain representations of the signal. At a 50% duty cycle, the square wave was symmetrical, while decreasing the duty cycle made the waveform narrower and more asymmetric, with the pulse being "on" for only 20% of the period and "off" for the remaining 80%. As the time-domain signal became narrower, the corresponding frequency spectrum widened. For the original (50%) duty cycle, only odd harmonics were present, whereas for the 20% duty cycle, both odd and even harmonics appeared. Additionally, the amplitudes of the harmonics for the 20% duty cycle were lower compared to those for the 50% duty cycle, likely because the narrower waveform contains less energy per cycle.

3.3. Evaluation Problem 3 – FFT of Multiple – Tone Sinusoidal Wave

The observed FFT spectrum for the third part of the experiment showed two distinct frequency peaks at approximately 9.900 kHz and 1.000 kHz. Since the input signal contained two frequencies, 10 kHz and 1 kHz, the FFT spectrum successfully identified both, demonstrating the linearity of the system. The slight deviation in the measured 10 kHz frequency can be attributed to the frequency scale not being sufficiently expanded to enable precise cursor-based measurement. Additionally, since the cursor function was adjusted manually, a minor human error during measurement could also have contributed to the discrepancy.

4. Conclusion

The experiment aimed to study different signals in terms of their Fourier coefficients and to gain a deeper understanding of the Fourier Transform, with particular focus on the implementation of the Fast Fourier Transform (FFT).

In the first part, the FFT of a single-tone sinusoidal wave was analyzed. It was observed that a signal containing a single frequency produced an FFT spectrum with a single dominant peak at that frequency. The comparison between the measured and simulated spectra showed good agreement, with minor deviations attributed to instrumental noise from the oscilloscope and measurement inaccuracies due to limited resolution.

The second part examined the FFT of a square wave with varying duty cycles. As the duty cycle decreased, the time-domain pulses became narrower, which resulted in a wider frequency

spectrum. Additionally, the amplitudes of the harmonic components decreased with lower duty cycles, likely because the narrower pulses carried less energy per cycle.

In the final part, the FFT of a multi-tone sinusoidal wave was studied. The resulting spectrum exhibited distinct peaks corresponding to each input frequency, confirming the linearity property of the FFT.

Overall, the experiment effectively demonstrated the relationship between time-domain and frequency-domain representations of signals, reinforcing the fundamental concepts of Fourier analysis and the practical use of FFT in signal processing.

5. References

- 20240829-co-520-b manual- U.Pagel, Page(35-64) http://uwp-cu-lab.my-board.org/02.0.adveelab/02.1.signalsys/20240829-co-520-b manual.pdf 2.

6. Appendix

6.1. Prelab

6.1.1 Prelab Problem 1 – The Sampling Theorem

1. Analog signals are usually passed through a low-pass filter prior to sampling. Why is this necessary?

Before sampling, an analog signal is passed through a low-pass filter to prevent aliasing. Aliasing occurs when the sampling rate is lower than the Nyquist frequency, causing higher frequency components to fold back into the lower frequency range. This overlap distorts the reconstructed signal and results in inaccurate representation of the original waveform.

2. What is the minimum sampling frequency for a pure sine wave input at 3KHz? Assume that the signal can be completely reconstructed.

The minimum required sampling frequency must be at least twice the highest frequency component of the signal. Since the maximum frequency here is 3 kHz, the sampling frequency should therefore be:

$$Sampling\ Frequency = 2 * 3 = 6\ kHz$$

3. What is the Nyquist frequency?

The Nyquist frequency is defined as half of the sampling frequency. It represents the highest frequency component that can be accurately captured and reconstructed during the sampling process without introducing aliasing.

4. What are the resulting frequencies for the following input sinusoids 500Hz, 2.5KHz, 5KHz and 5.5KHz if the signals are sampled by a sampling frequency of 5KHz?

It is important to note that for any input frequency higher than the Nyquist frequency, the resulting (or aliased) frequency can be determined using the relation:

$$Resulting\ Frequency = |Input - Sampling|$$

- 500 Hz
  Input Frequenzy < Nyquist Frequency
  Result = 500 Hz
- 2.5 Khz
  Input Frequency < Nyquist Frequency
  Result = 2.5 kHz
- 5 kHz
  Result = |5000 - 5000| = 0 Hz
- 5.5 kHz = |5500 - 50000| = 500 Hz

5. Mention three frequencies of signal that alias to a 7Hz signal. The signal is sampled by a constant 30 Hz sampling frequency.

The frequencies that alias to a 7 Hz signal when sampled at a 30 Hz sampling rate can be found using the formula:

$$f = |n * f_s + f_a|$$

Where :
n is any integer, $f_s = 30\ Hz, f_a = 7\ Hz$
Substituting the values gives:

$$For\ n = 1: f = |1 * 30 + 7| = 37\ Hz$$
$$For\ n = 2: f = |2 * 30 + 7| = 67\ Hz$$
$$For\ n = 3: f = |3 * 30 + 7| = 97\ Hz$$

Hence, the three frequencies that alias to 7 Hz are 37 Hz**,** 67 Hz, and 97 Hz.

6.1.2 Prelab Problem 2 – Impulse Train Sampling and Real Sampling

Consider the sampling schema shown in figure (8.2). The input signal x(t) is given by a sine function, with an amplitude of 5 V peak and a frequency of 50 Hz. The sampling signal p(t) is represented by a unity impulse train. Use an overall sampling rate of 100 k samples/s) for the whole problem.

1. Carry out simulations for the following cases:
   a. Under Sampling (use 48 Hz)

b. Nyquist Sampling
c. Over Sampling (use 1000 Hz)

Use the command subplot to visualize the continuous signal x(t), the sampling signal p(t) and the result for each of these cases.

MATLAB Code for this part:

```
clear; clc;
%%

% Input signal
Fs = 100e3;
t = 0:1/Fs:0.1; % 0.1 to show 5 cycles as one cycle is 0.02
x_t = 5*sin(2*pi*50*t);

% Sampling frequencies
Fs_under = 48;
Fs_nyq = 2*50; % nyquist frequnecy = 2 times the max frequency
Fs_over = 1000;

% Impulse train for each sampling frequency
p_under = zeros(size(t));
p_under(1:round(Fs/Fs_under):end) = 1; % Set impulses spaced by Fs_under

p_nyq = zeros(size(t));
p_nyq(1:round(Fs/Fs_nyq):end) = 1; % Set impulses spaced by Fs_nyquist

p_over = zeros(size(t));
p_over(1:round(Fs/Fs_over):end) = 1; % Set impulses spaced by Fs_over

% Output signals
x_under = x_t .* p_under;
x_nyq = x_t .* p_nyq;
x_over = x_t .* p_over;

% Result Plots
%% Under sampling
figure;
% Plot for Under Sampling (48 Hz)
subplot(3, 1, 1);
plot(t, x_t, 'b', 'LineWidth', 1.5);      % Continuous signal
title('Input signal');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,2);
stem(t, p_under, 'k', 'filled');          % Impulse train
title('Impulse at sampling rate 48Hz');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,3);
stem(t, x_under, 'r', 'filled');          % Sampled signal
title('Output signal (Sampling rate = 48Hz)');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

%% Nyquist Sampling
figure;
% Plot for Nyqusit Sampling
subplot(3, 1, 1);
plot(t, x_t, 'b', 'LineWidth', 1.5);      % Continuous signal
title('Input signal');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,2);
stem(t, p_nyq, 'k', 'filled');            % Impulse train
title('Impulse at Nyquist Sampling');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,3);
stem(t, x_nyq, 'r', 'filled');          % Sampled signal
title('Output signal (Nyquist Sampling)');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

%% Over sampling
figure;
% Plot for Over Sampling (1000 Hz)
subplot(3, 1, 1);
plot(t, x_t, 'b', 'LineWidth', 1.5);      % Continuous signal
title('Input signal');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,2);
stem(t, p_over, 'k', 'filled');           % Impulse train
title('Impulse at sampling rate 1000Hz');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,3);
stem(t, x_over, 'r', 'filled');           % Sampled signal
title('Output signal (Sampling rate = 1000Hz)');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;
```

The following figures were obtained,

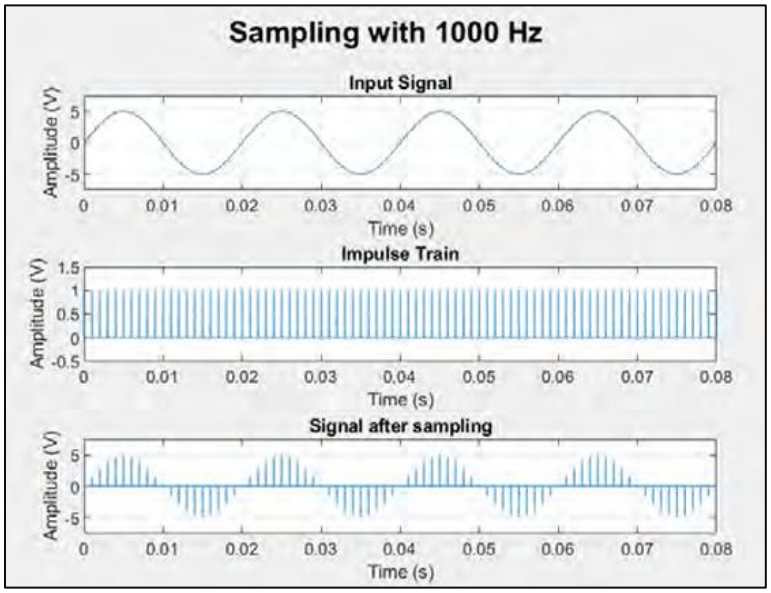


Figure 22. Oversampling

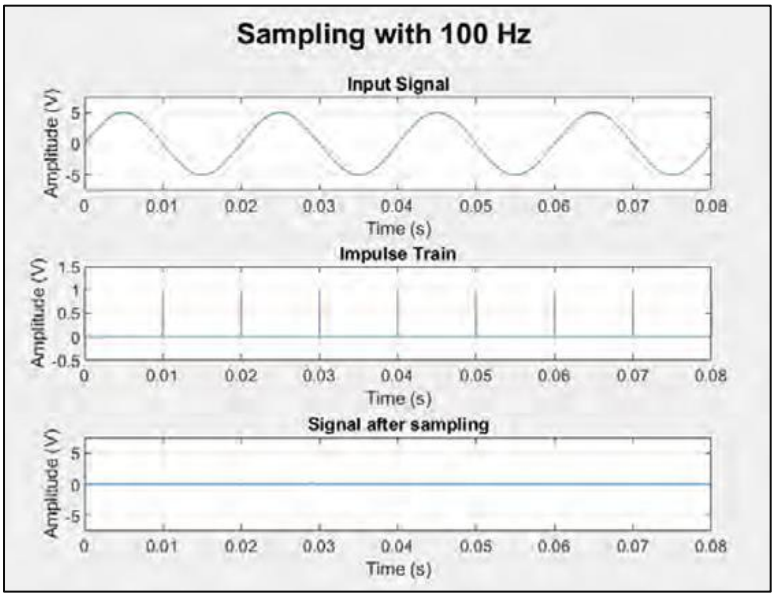


Figure 23. Nyquist Sampling

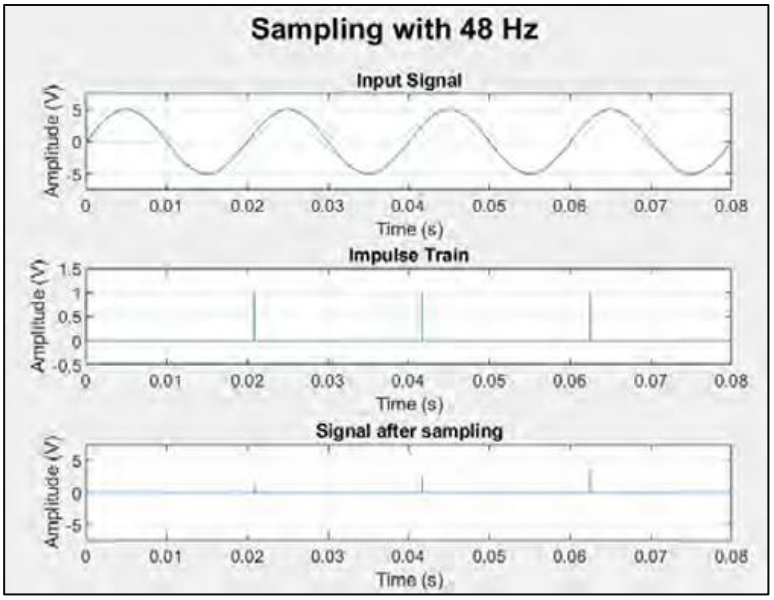


Figure 24. Under Sampling

2. The signal x(t) should be sampled by a rectangular pulse train. Modify the sampling function p(t), so that the width of the sampling pulse is 50% of the sampling period. Carry out simulations for the following cases:

a. Under Sampling
b. Nyquist Sampling
c. Over Sampling

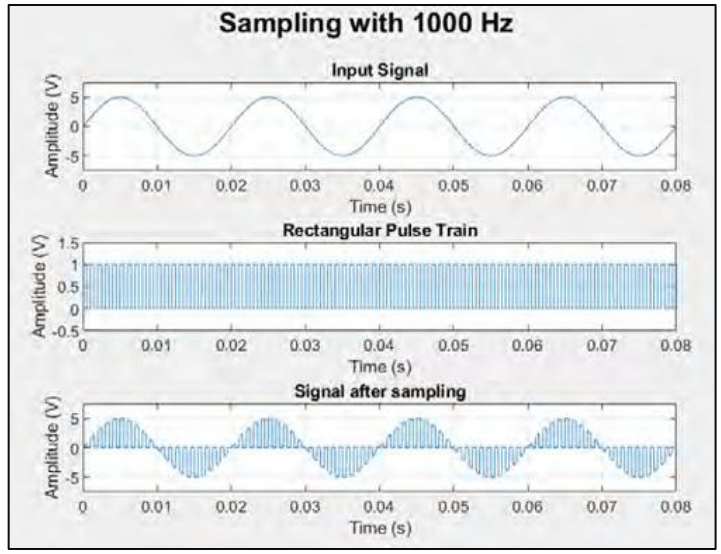
Use the same sampling rates and the same plot setup as before.

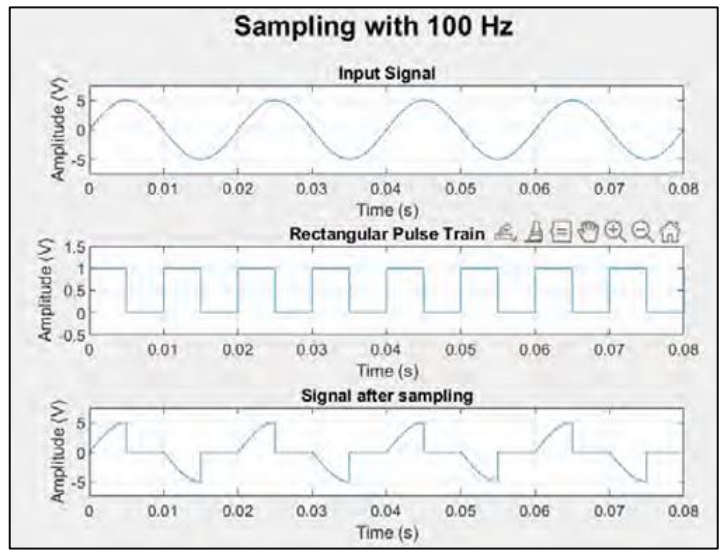
MATLAB Code for this part:

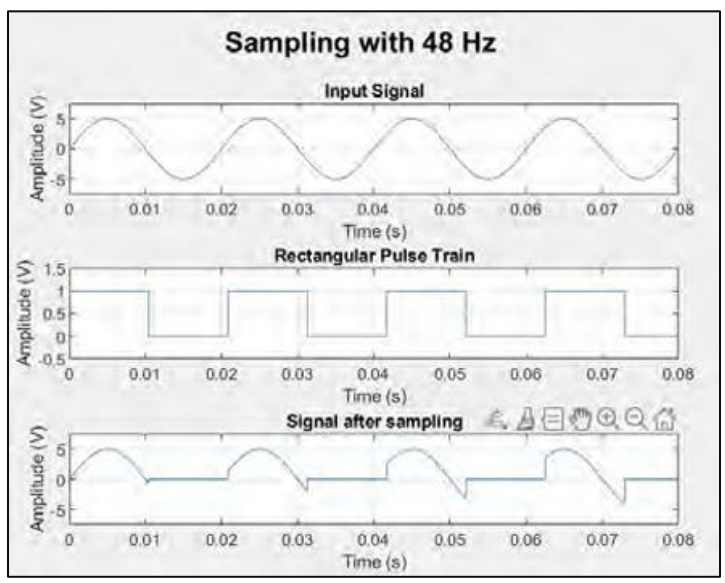
```
clear; clc;
%%

% Input signal
Fs = 100e3;
t = 0:1/Fs:0.1; % 0.1 to show 5 cycles as one cycle is
   0.02
x_t = 5*sin(2*pi*50*t);

% Sampling frequencies
Fs_under = 48;
Fs_nyq = 2*50; % nyquist frequnecy = 2 times the max
   frequency
Fs_over = 1000;

```

```
% Impulse train for each sampling frequency
pulse_width_under = 1/(2*Fs_under);
pulse_width_nyq = 1/(2*Fs_nyq);
pulse_width_over = 1/(2*Fs_over);

p_under = zeros(size(t)); % Sampling Pulse for Under sampling
for n = 0 : floor(0.1*Fs_under)
    pulse = rectpuls(t - n/Fs_under, pulse_width_under); %forming a pulse of half the width and time shifting it
    p_under = p_under + pulse;
end

p_nyq = zeros(size(t)); % Sampling Pulse for Nyquist sampling
for n = 0 : floor(0.1*Fs_nyq) -1
    pulse = rectpuls(t - n/Fs_nyq, pulse_width_nyq);
    p_nyq = p_nyq + pulse;
end

p_over = zeros(size(t)); % Sampling Pulse for Nyquist sampling
for n = 0 : floor(0.1*Fs_over) -1
    pulse = rectpuls(t - n/Fs_over, pulse_width_over);
    p_over = p_over + pulse;
end

% Output signals
x_under = x_t .* p_under;
x_nyq = x_t .* p_nyq;
x_over = x_t .* p_over;

% Result Plots
%% Under sampling
figure;
% Plot for Under Sampling (48 Hz)
subplot(3, 1, 1);
plot(t, x_t, 'b', 'LineWidth', 1.5);      % Continuous signal
title('Input signal');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,2);
plot(t, p_under, 'k');                    % Sampling
```

```
Pulse
title('Sampling pulse at sampling rate 48Hz');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,3);
plot(t, x_under, 'r');                    % Sampled signal
title('Output signal (Sampling rate = 48Hz)');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

%% Nyquist Sampling
figure;
% Plot for Nyqusit Sampling
subplot(3, 1, 1);
plot(t, x_t, 'b', 'LineWidth', 1.5);      % Continuous signal
title('Input signal');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,2);
plot(t, p_nyq, 'k');                      % Pulse train
title('Sampling pulse at Nyquist Sampling');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,3);
plot(t, x_nyq, 'r');                      % Sampled signal
title('Output signal (Nyquist Sampling)');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

%% Over sampling
figure;
% Plot for Over Sampling (1000 Hz)
subplot(3, 1, 1);
plot(t, x_t, 'b', 'LineWidth', 1.5);      % Continuous signal
title('Input signal');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,2);
plot(t, p_over, 'k');                     % Pulse train
title('Sampling pulse train at sampling rate 1000Hz');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;

subplot(3,1,3);
plot(t, x_over, 'r');                     % Sampled signal
title('Output signal (Sampling rate = 1000Hz)');
xlabel('Time (s)');
ylabel('Amplitude (V)');
grid on;
```







6.1.3 Prelab Problem 3 – Sampling Using a Sampling Bridge

Modify the circuit in figure (8.6) in such a way that a single sampling source can be used to sample the input signal.

1. Sketch the modified circuit.

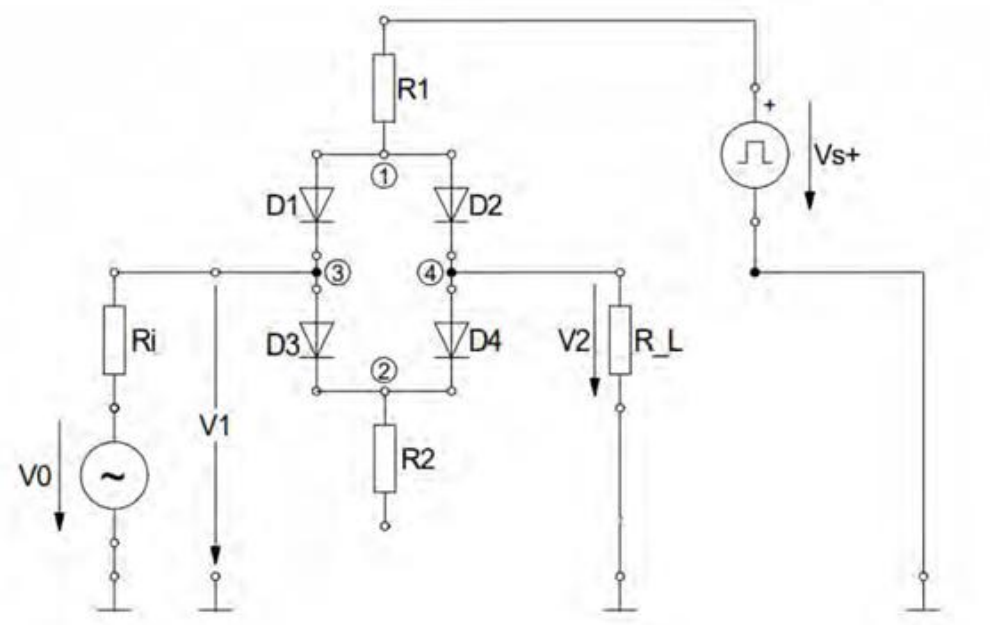


Figure 25. Modified Circuit

2. Explain the operation of the modified circuit.

In the original circuit, two square waves—a positive and a negative—were used as the sampling sources. The positive wave sampled the positive half of the input signal, while the negative wave sampled the negative half. In the modified circuit, only the positive sampling source remains. As a result, the circuit can sample only the positive half of the input signal, leading to half-wave sampling, where the negative portion of the signal is not processed.